DAT 471 – Computational Techniques for Large-Scale Data

Module 1

Group 6

Name	Programme	Email
Gabriela Castillo Avila	MPHPC	avilag@student.chalmers.se
Mahnoor Ali Cheema	MPDSC	mahnoora@student.chalmers.se

04 April 2026

Problem 1 - Information of Computers

Following are the results obtained when we execute assignment1_problem1.sh file for login node (Minerva) and Two SLURM nodes which in our case, **io** and **neptune**.

Hostname	minerva
CPU Model and Frequency	Model name: Intel(R) Xeon(R) Silver 4410Y CPU scaling MHz: 36% CPU max MHz: 3900.0000 CPU min MHz: 800.0000
CPU Sockets / Cores / Threads	Threads per core: 2 Cores per socket: 12 Sockets: 1
CPU Architecture	x86_64
Cache Line Size	64 bytes
Cache Sizes (L1, L2, L3)	L1d cache: 576 KiB (12 instances) L1i cache: 384 KiB (12 instances) L2 cache: 24 MiB (12 instances) L3 cache: 30 MiB (1 instance)
System RAM	Total: 125 GiB Used: 107 GiB Free: 12 GiB Available: 17 GiB
GPU Count and Model	No NVIDIA GPUs detected
GPU RAM	N/A
Filesystem Type of /data	Type: zfs
Disk Space on /data	Size: 59T Used: 51T Available: 7.8T Usage: 87%
Linux Kernel Version	6.8.0-88-generic
Linux Distribution	Ubuntu 24.04.3 LTS (Noble Numbat)
Default Python 3 Interpreter	/usr/bin/python3 Python 3.12.3

Hostname	io
CPU Model and Frequency	Model name: INTEL(R) XEON(R) GOLD 6548N
CPU Sockets / Cores / Threads	Threads per core: 2 Cores per socket: 32 Sockets: 2
CPU Architecture	x86_64
Cache Line Size	64 bytes
Cache Sizes (L1, L2, L3)	L1d cache: 3 MiB (64 instances) L1i cache: 2 MiB (64 instances) L2 cache: 128 MiB (64 instances) L3 cache: 120 MiB (2 instances)
System RAM	Total: 503 GiB Used: 43 GiB Free: 56 GiB Available: 460 GiB
GPU Count and Model	8 × NVIDIA L4 GPUs Driver Version: 565.57.01 CUDA Version: 12.7
GPU RAM	Each GPU: 23,034 MiB
Filesystem Type of /data	nfs4
Disk Space on /data	Size: 59T Used: 51T Available: 7.8T Usage: 87%
Linux Kernel Version	6.8.0-101-generic
Linux Distribution	Ubuntu 24.04.4 LTS (Noble Numbat)
Default Python 3 Interpreter	/usr/bin/python3 Python 3.12.3

Hostname	neptune
CPU Model and Frequency	Model name: INTEL(R) XEON(R) GOLD 6548N
CPU Sockets / Cores / Threads	Threads per core: 2 Cores per socket: 32 Sockets: 2
CPU Architecture	x86_64
Cache Line Size	64 bytes
Cache Sizes (L1, L2, L3)	L1d cache: 3 MiB (64 instances) L1i cache: 2 MiB (64 instances) L2 cache: 128 MiB (64 instances) L3 cache: 120 MiB (2 instances)
System RAM	Total: 503 GiB Used: 11 GiB Free: 63 GiB Available: 491 GiB
GPU Count and Model	4 × NVIDIA L40S GPUs Driver Version: 565.57.01 CUDA Version: 12.7
GPU RAM	Each GPU: 46,068 MiB
Filesystem Type of /data	nfs4
Disk Space on /data	Size: 59T Used: 51T Available: 7.8T Usage: 87%
Linux Kernel Version	6.8.0-101-generic
Linux Distribution	Ubuntu 24.04.3 LTS (Noble Numbat)
Default Python 3 Interpreter	/usr/bin/python3 Python 3.12.3

Problem2 - Apptainer

Part a: Create SLURM batch script that runs batch script on any node in clusyter to get information about container

When we executed assignment1_problem2a.sh batch script on **callisto** node we got the following results:

Running Apptainer Container Information	Date: Thu Apr 2 13:15:43 CEST 2026 Node: callisto SLURM Job ID: 167612
Kernel Version Inside Container	6.8.0-101-generic
Python 3 Version Inside Container	Python 3.13.12
CPU Model Inside Container	Model name: INTEL(R) XEON(R) GOLD 6548N

Part b:

To run the script within the container, the following command was executed:

```
apptainer exec --bind
/data:/data
/data/.../assignment1.sif
python3 /opt/mystery.py
/data/.../bike_sharing_hourly.csv
```

The output of running this command was:

```
17
```

To determine what question this output answers, the following command was used to inspect the script:

```
apptainer exec
/data/.../assignment1.sif
cat /opt/mystery.py
```

Producing the following output:

```
mystery.py
import sys
import pandas as pd

if len(sys.argv) != 2:
 sys.stderr.write(f'Usage: {sys.argv[0]} <csv_file>\n')
 sys.exit(1)

df = pd.read_csv(sys.argv[1])

avg_hourly_count_good_weather =
df.query('weathersit == 1').groupby('hr')['cnt'].mean()

avg_hourly_count_bad_weather =
df.query('weathersit >= 3').groupby('hr')['cnt'].mean()

worst_hourly_penalty =
(avg_hourly_count_good_weather -
```

```
avg_hourly_count_bad_weather).argmax()

print(worst_hourly_penalty)
```

Inspecting the output representing the code within the script, we can see that it computes the average hourly bike rental count during either good or bad weather.
The output of the script, the number 17, therefore is answering the question:

```
At which hour is the decrease in rental rate the largest, during bad
weather?
```

Part c:

To get the correct names of the columns in order to answer the provided questions the following code was used

```
head -n 1 /data/.../bike_sharing_hourly.csv
```

This command retrieved the column names used in the dataset.

```
instant,dteday,season,yr,mnth,hr,holiday,weekday,workingday,
weathersit,temp,atemp,hum,windspeed,casual,registered,cnt
```

With the correct column names the following questions could be answered using the following SQL queries:

- **How many rows of data there are in the file?**

  ```
  SELECT COUNT(*) AS row_count FROM hour
  ```

  Answer to the question:

  ```
  row_count
  17379
  ```

- **What was the average hourly count of bike rentals?**

  ```
  SELECT AVG(cnt) AS avg_hourly FROM hour
  ```

  Answer to the question:

  ```
  avg_hourly
  189.463088
  ```

- **Which were the top-5 busiest hours in terms of average bike rentals?**

  ```
  SELECT hr,AVG(cnt) AS avg_hourly
  FROM hour GROUP BY hr ORDER BY avg_hourly DESC LIMIT 5
  ```

  Answer to the question:

```
hr  avg_hourly
17  461.452055
18  425.510989
8  359.011004
16  311.983562
19  311.523352
```

- **What was the average daily count of bike rentals in the month of January 2012?**

```
SELECT AVG(daily_count) AS avg_daily
FROM ( SELECT dteday, SUM(cnt) AS daily_count FROM hour
WHERE yr=1 AND mnth=1 GROUP BY dteday)
```

Answer to the question:

```
avg_daily
3120.774194
```